E-commerce Order Fulfillment & Delivery Analytics

Submitted by:

Thiageshwaran S

ADMM

Technology And Transformation

March 2026

E-Commerce Order Fulfillment & Delivery Analytics

SAP ERP Integration with SQL Data Warehouse & Power BI

1. BUSINESS PROBLEM STATEMENT

1.1 Background

ShopX is a rapidly growing e-commerce enterprise operating across multiple regions with a diverse customer base. The company uses SAP ERP to manage core operational processes including:

- ✓ Sales Orders (Order Management System – OMS)
- ✓ Shipping and Logistics
- ✓ Inventory Management
- ✓ Customer Master Data
- ✓ Carrier/Vendor Management

While SAP ERP efficiently manages transactional workflows, it lacks advanced analytical capabilities required for performance monitoring, operational optimization, and executive-level reporting.

ShopX faced several operational challenges:

- ✓ Limited visibility into order processing delays
- ✓ Inability to measure warehouse fulfillment efficiency
- ✓ No structured tracking of SLA compliance
- ✓ Lack of consolidated KPI dashboards
- ✓ Poor carrier utilization visibility
- ✓ Difficulty identifying operational bottlenecks

To overcome these challenges, the company initiated a project to integrate SAP ERP data into a SQL-based Data Warehouse and develop analytical dashboards using Power BI.

1.2 Case Study Scope

The scope of this case study includes the design and implementation of an end-to-end analytics solution integrating SAP ERP data with a Data Warehouse and Business Intelligence layer.

The solution integrates the following components:

1.2.1 Order Management System (OMS)

Tracks orders from the moment they are placed by the customer until they are shipped.

1.2.2 Shipping & Delivery Analytics

Monitors logistics activities including shipment creation, shipping status, carrier allocation, and SLA compliance.

1.2.3 Customer Analytics

Analyze customer behavior, order frequency, operational impact, and segmentation-based performance.

1.3 Primary Objectives

The primary objective of this solution is to enable ShopX to track and optimize:

- ✓ Order processing times
- ✓ Fulfillment efficiency
- ✓ Shipment activity and load
- ✓ SLA compliance
- ✓ Operational bottlenecks
- ✓ Customer and sales segmentation insights

2. OBJECTIVES – KPIs AND GOALS

2.1 Key Performance Indicators (KPIs)

The following KPIs were designed and implemented:

2.1.1 Total Orders

Measures overall business demand.

2.1.2 Total Shipments

Measures operational execution volume.

2.1.3 Average Processing Time

Measures internal operational speed.

Processing Time = Shipment Date – Order Date

2.1.4 Fulfillment Efficiency (%)

Percentage of orders processed within SLA (≤ 2 days).

2.1.5 Processing SLA Compliance (%)

Percentage of shipments meeting internal processing standards.

2.1.6 On-Time Delivery Rate (%)

Measures shipment compliance against defined business rules.

2.1.7 Shipment per Order Ratio

Indicates operational complexity and order splitting behavior.

2.1.8 Average Items per Order

Measures basket size and order structure complexity.

2.2 Strategic Goals

The project aims to:

- ✓ Improve warehouse operational efficiency
- ✓ Reduce processing delays
- ✓ Monitor SLA compliance
- ✓ Optimize shipment allocation
- ✓ Analyze operational complexity
- ✓ Support executive decision-making

3. ETL PROCESS USING PYTHON AND SQL SERVER

A structured industry-standard ETL workflow was implemented using Python (Jupyter Notebook) and Microsoft SQL Server.

3.1 Extraction Phase

Tools Used

- Python (Pandas)
- Jupyter Notebook (VS Code)
- openpyxl
- SQLAlchemy
- pyodbc

Steps Performed

1. Created structured project directory.
2. Loaded Excel dataset using Pandas.
3. Extracted the following sheets:
    - ✓ VBAK (Order Header)
    - ✓ VBAP (Order Items)
    - ✓ LIKP (Delivery Header)
    - ✓ LIPS (Delivery Items)
    - ✓ KNA1 (Customers)
    - ✓ LFA1 (Carriers)
    - ✓ VTTK (Shipment Header)
    - ✓ VTTP (Shipment Items)
4. Conducted initial data inspection (shape, datatypes, preview).

Initial Data Inspection

```python
#check dimensions
print("VBAK Shape:", vbak.shape)
print("VBAP Shape:", vbap.shape)
print("LIKP Shape:", likp.shape)
print("LIPS Shape:", lips.shape)
print("KNA1 Shape:", kna1.shape)
print("LFA1 Shape:", lfa1.shape)
print("VTTK Shape:", vttk.shape)
print("VTTP Shape:", vttp.shape)
```

```
VBAK Shape: (22, 8)
VBAP Shape: (23, 7)
LIKP Shape: (20, 10)
LIPS Shape: (21, 10)
KNA1 Shape: (30, 15)
LFA1 Shape: (30, 12)
VTTK Shape: (20, 10)
VTTP Shape: (21, 10)
```

Standardize Date Columns

```python
#Orders Date
vbak['ERDAT'] = pd.to_datetime(vbak['ERDAT'], errors='coerce')
# Shipment Date
likp['WADAT'] = pd.to_datetime(likp['WADAT'], errors='coerce')
# Delivery Date
lips['LFDAT'] = pd.to_datetime(lips['LFDAT'], errors='coerce')
```

```python
dataframes = {
    "VBAK": vbak,
    "VBAP": vbap,
    "LIKP": likp,
    "LIPS": lips,
    "KNA1": kna1,
    "LFA1": lfa1,
    "VTTK": vttk,
    "VTTP": vttp
}

for name, df in dataframes.items():
    print(f"\n{name} Columns:")
    print(df.columns.tolist())
```

```python
vbak = pd.read_excel(xls, sheet_name='VBAK')

vbap = pd.read_excel(xls, sheet_name='VBAP')

likp = pd.read_excel(xls, sheet_name='LIKP')

lips = pd.read_excel(xls, sheet_name='LIPS')

kna1 = pd.read_excel(xls, sheet_name='KNA1')

lfa1 = pd.read_excel(xls, sheet_name='LFA1')

vttk = pd.read_excel(xls, sheet_name='VTTK')

vttp = pd.read_excel(xls, sheet_name='VTTP')
```

3.2 Data Validation

The following validations were performed:

- ✓ Missing key field checks
- ✓ Duplicate detection using composite keys
- ✓ Referential integrity checks
- ✓ Date validation and normalization
- ✓ Negative quantity and pricing checks
- ✓ Logical date sequence verification

Dataset Issue Identified

Delivery Date was equal to Shipment Date in the dataset.
This resulted in Delivery Time = 0, making the Average Delivery Time KPI analytically irrelevant.

Resolution

Average Delivery Time and related DAX measures were removed from the KPI layer and replaced with more meaningful operational metrics.

Duplicate Detection

```
vbak['Sales Document'].duplicated().sum()
```

```
np.int64(0)
```

```
vbap.duplicated(subset=['Sales Document','Item Number']).sum()
```

```
np.int64(0)
```

```
likp['Delivery Number'].duplicated().sum()
```

```
np.int64(0)
```

```
lips.duplicated(subset=['Delivery Number','Item Number']).sum()
```

```
np.int64(0)
```

Referential Integrity Checks

```
#Items Without Orders
invalid_items = vbap[~vbap['Sales Document'].isin(vbak['Sales Document'])]

invalid_items
```

Sales Document	Item Number	Material Number	Quantity	Net Price	Item Status	Delivery Date

```
#Deliveries Without Orders
invalid_deliveries = likp[~likp['Sales Document'].isin(vbak['Sales Document'])]

invalid_deliveries
```

Delivery Number	Delivery Date	Sales Document	Shipping Point	Shipping Type	Delivery Status

```
#Delivery Items Without Deliveries
invalid_delivery_items = lips[~lips['Delivery Number'].isin(likp['Delivery Number'])]

invalid_delivery_items
```

Delivery Number	Item Number	Material Number	Delivered Quantity	Net Price	Delivery Status

Date Validation

```
vbak['Order Date'] = pd.to_datetime(vbak['Order Date'], errors='coerce')
likp['Delivery Date'] = pd.to_datetime(likp['Delivery Date'], errors='coerce')
lips['Delivery Date'] = pd.to_datetime(lips['Delivery Date'], errors='coerce')
vttk['Shipment Date'] = pd.to_datetime(vttk['Shipment Date'], errors='coerce')
vttp['Shipment Date'] = pd.to_datetime(vttp['Shipment Date'], errors='coerce')
```

Detect Invalid Dates

```
vbak[vbak['Order Date'].isnull()]
```

Sales Document	Order Date	Customer ID	Order Type	Sales Organization	Distribution Channel

Logical Date Consistency

```
#Shipment Before Order?
merged_dates = pd.merge(
    vbak,
    vttk,
    on='Sales Document',
    how='inner'
)
```

DATA PROFILING & VALIDATION

Missing Value Analysis

```
vbak[['Sales Document', 'Order Date', 'Customer ID']].isnull().sum()
```

```
Sales Document    0
Order Date        0
Customer ID       0
dtype: int64
```

```
vbap[['Sales Document', 'Item Number', 'Material Number']].isnull().sum()
```

```
Sales Document     0
Item Number        0
Material Number    0
dtype: int64
```

```
likp[['Delivery Number', 'Sales Document', 'Customer ID']].isnull().sum()
```

```
Delivery Number    0
Sales Document     0
Customer ID        0
```

3.3 Transformation Phase

Raw SAP-style tables were transformed into business-friendly Data Warehouse tables.

S.No	Raw Table	Transformed Table
1	VBAK	DW_Orders
2	VBAP	DW_Order_Items
3	KNA1	DW_Customers
4	LFA1	DW_Carriers
5	LIKP + VTTK	DW_Shipments
6	LIPS + VTTP	DW_Shipment_Items

Standardization Applied

- ✓ Snake_case column naming
- ✓ Composite key preservation
- ✓ Duplicate removal
- ✓ Date normalization
- ✓ Business semantic clarity
- ✓ Removal of redundant fields

3.4 Loading into SQL Server

Using SQLAlchemy, transformed DataFrames were loaded into:

Database Name: ShopX_DW

SOURCE CODE:

```
CREATE DATABASE ShopX_DW;
use ShopX_DW;
SELECT COUNT(*) FROM DW_Orders;
SELECT COUNT(*) FROM DW_Order_Items;
SELECT COUNT(*) FROM DW_Customers;
SELECT COUNT(*) FROM DW_Shipments;

-- Order Processing Time
SELECT
   o.order_id,
   o.order_date,
   s.shipment_number,
   s.shipment_date,
   DATEDIFF(DAY, o.order_date, s.shipment_date) AS processing_time_days
FROM DW_Orders o
JOIN DW_Shipments s
   ON o.order_id = s.order_id;

-- Delivery Time
SELECT
   s.shipment_number,
   s.shipment_date,
   si.delivery_date,
   DATEDIFF(DAY, s.shipment_date, si.delivery_date) AS delivery_time_days
FROM DW_Shipments s
JOIN DW_Shipment_Items si
   ON s.shipment_number = si.shipment_number;

   -- Add On-Time Flag
   SELECT
   s.shipment_number,
   DATEDIFF(DAY, s.shipment_date, si.delivery_date) AS delivery_time_days,
   CASE
```

```sql
        WHEN DATEDIFF(DAY, s.shipment_date, si.delivery_date) <= 3
        THEN 'On-Time'
        ELSE 'Delayed'
    END AS on_time_flag
FROM DW_Shipments s
JOIN DW_Shipment_Items si
    ON s.shipment_number = si.shipment_number;

    -- Detect Negative or Illogical Cases
    SELECT *
FROM DW_Shipments s
JOIN DW_Shipment_Items si
    ON s.shipment_number = si.shipment_number
WHERE si.delivery_date < s.shipment_date;

-- Final Analytics Table
CREATE TABLE DW_Delivery_Analytics (
    order_id VARCHAR(50),
    shipment_number VARCHAR(50),
    customer_id VARCHAR(50),
    carrier_id VARCHAR(50),
    order_date DATE,
    shipment_date DATE,
    delivery_date DATE,
    processing_time_days INT,
    delivery_time_days INT,
    on_time_flag VARCHAR(20)
);

-- Insert Data

INSERT INTO DW_Delivery_Analytics
SELECT
    o.order_id,
    s.shipment_number,
    o.customer_id,
    s.carrier_id,
    o.order_date,
    s.shipment_date,
    si.delivery_date,
    DATEDIFF(DAY, o.order_date, s.shipment_date) AS processing_time_days,
    DATEDIFF(DAY, s.shipment_date, si.delivery_date) AS delivery_time_days,
    CASE
        WHEN DATEDIFF(DAY, s.shipment_date, si.delivery_date) <= 3
        THEN 'On-Time'
        ELSE 'Delayed'
    END AS on_time_flag
FROM DW_Orders o
JOIN DW_Shipments s
    ON o.order_id = s.order_id
JOIN DW_Shipment_Items si
    ON s.shipment_number = si.shipment_number;

    -- Aggregate KPIs

    --Average Processing Time
    SELECT AVG(processing_time_days) AS avg_processing_time
FROM DW_Delivery_Analytics;

-- Average Delivery Time
SELECT AVG(delivery_time_days) AS avg_delivery_time
FROM DW_Delivery_Analytics;
```

```sql
-- On-Time Delivery Rate
SELECT
   CAST(
      SUM(CASE WHEN on_time_flag = 'On-Time' THEN 1 ELSE 0 END)
      * 100.0 / COUNT(*)
   AS DECIMAL(5,2)) AS on_time_delivery_rate_percent
FROM DW_Delivery_Analytics;

-- Carrier Performance
SELECT
   carrier_id,
   AVG(delivery_time_days) AS avg_delivery_time,
   COUNT(*) AS total_shipments
FROM DW_Delivery_Analytics
GROUP BY carrier_id
ORDER BY avg_delivery_time;

-- SQL to Calculate Efficiency Per Order
SELECT
   order_id,
   processing_time_days,
   CASE
      WHEN processing_time_days <= 2
      THEN 'Efficient'
      ELSE 'Delayed Processing'
   END AS fulfillment_status
FROM DW_Delivery_Analytics;

-- Calculate Fulfillment Efficiency %
SELECT
   CAST(
      SUM(CASE WHEN processing_time_days <= 2 THEN 1 ELSE 0 END)
      * 100.0 / COUNT(*)
   AS DECIMAL(5,2))
   AS fulfillment_efficiency_percent
FROM DW_Delivery_Analytics;

-- Fulfillment Efficiency by Month
SELECT
   FORMAT(order_date, 'yyyy-MM') AS order_month,
   CAST(
      SUM(CASE WHEN processing_time_days <= 2 THEN 1 ELSE 0 END)
      * 100.0 / COUNT(*)
   AS DECIMAL(5,2))
   AS fulfillment_efficiency_percent
FROM DW_Delivery_Analytics
GROUP BY FORMAT(order_date, 'yyyy-MM')
ORDER BY order_month;

-- Fulfillment Efficiency by Shipping Point

SELECT
   s.shipping_point,
   CAST(
      SUM(CASE WHEN a.processing_time_days <= 2 THEN 1 ELSE 0 END)
      * 100.0 / COUNT(*)
   AS DECIMAL(5,2))
   AS fulfillment_efficiency_percent
FROM DW_Delivery_Analytics a
JOIN DW_Shipments s
   ON a.shipment_number = s.shipment_number
```

```
GROUP BY s.shipping_point
ORDER BY fulfillment_efficiency_percent DESC;
```

	avg_processing_time
1	7

	avg_delivery_time
1	0

	on_time_delivery_rate_percent
1	100.00

	carrier_id	avg_delivery_time	total_shipments
1	Carrier1	0	9
2	Carrier2	0	6
3	Carrier3	0	6

	order_id	processing_time_days	fulfillment_status
1	1000001	9	Delayed Processing
2	1000001	9	Delayed Processing
3	1000002	9	Delayed Processing
4	1000003	7	Delayed Processing

	order_month	fulfillment_efficiency_percent
1	2025-02	0.00

	shipping_point	fulfillment_efficiency_percent
1	SP001	0.00
2	SP002	0.00
3	SP003	0.00

Tables created:

- ✓ DW_Customers
- ✓ DW_Orders
- ✓ DW_Order_Items
- ✓ DW_Shipments
- ✓ DW_Shipment_Items
- ✓ DW_Carriers
- ✓ DW_Delivery_Analytics

Row count validations were performed post-load to ensure data integrity.

Transform Orders (VBAK → DW_Orders)

```python
dw_orders = vbak.copy()

dw_orders = dw_orders.rename(columns={
    'Sales Document': 'order_id',
    'Order Date': 'order_date',
    'Customer ID': 'customer_id',
    'Order Type': 'order_type',
    'Sales Organization': 'sales_organization',
    'Distribution Channel': 'distribution_channel',
    'Division': 'division',
    'Order Status': 'order_status'
})

dw_orders = dw_orders.drop_duplicates(subset=['order_id'])
dw_orders = dw_orders.reset_index(drop=True)

dw_orders.head()
```

Transform Order Items (VBAP → DW_Order_Items)

```python
dw_order_items = vbap.copy()

dw_order_items = dw_order_items.rename(columns={
    'Sales Document': 'order_id',
    'Item Number': 'item_number',
    'Material Number': 'material_number',
    'Quantity': 'quantity',
    'Net Price': 'net_price',
    'Item Status': 'item_status',
    'Delivery Date': 'requested_delivery_date'
})

dw_order_items = dw_order_items.drop_duplicates(
    subset=['order_id', 'item_number']
)

dw_order_items = dw_order_items.reset_index(drop=True)

dw_order_items.head()
```

	order_id	item_number	material_number	quantity	net_price	item_status	requested_delivery_date
0	1000001	10	MAT001	100	50	Open	2025-02-08
1	1000001	20	MAT002	200	30	Open	2025-02-08
2	1000002	10	MAT003	150	40	Delivered	2025-02-09
3	1000003	10	MAT001	300	50	Open	2025-02-10
4	1000004	10	MAT004	500	25	Closed	2025-02-12

Transform Carriers (LFA1 → DW_Carriers)

```python
dw_carriers = lfa1.copy()

dw_carriers = dw_carriers.rename(columns={
    'Vendor Number': 'carrier_id',
    'Vendor Name': 'carrier_name',
    'Country': 'country',
    'Region': 'region',
    'City': 'city',
    'Postal Code': 'postal_code',
    'Street Address': 'street_address',
    'Phone Number': 'phone_number',
    'Email Address': 'email',
    'Language': 'language',
    'Tax Number': 'tax_number',
    'Payment Terms': 'payment_terms'
})

dw_carriers = dw_carriers.drop_duplicates(subset=['carrier_id'])
dw_carriers = dw_carriers.reset_index(drop=True)

dw_carriers.head()
```

Transform Shipments (LIKP + VTTK)

```python
# Rename LIKP
likp_clean = likp.rename(columns={
    'Delivery Number': 'delivery_number',
    'Delivery Date': 'delivery_date',
    'Sales Document': 'order_id',
    'Shipping Point': 'shipping_point',
    'Shipping Type': 'shipping_type',
    'Delivery Status': 'delivery_status',
    'Shipping Status': 'shipping_status',
    'Route': 'route',
    'Delivery Priority': 'delivery_priority',
    'Customer ID': 'customer_id'
})

# Rename VTTK
vttk_clean = vttk.rename(columns={
    'Shipment Number': 'shipment_number',
    'Shipment Date': 'shipment_date',
    'Sales Document': 'order_id',
    'Delivery Number': 'delivery_number',
    'Shipping Point': 'shipping_point',
    'Carrier': 'carrier_id',
    'Shipment Status': 'shipment_status',
    'Route': 'route',
    'Shipping Type': 'shipping_type',
    'Customer ID': 'customer_id'
})
```

Transform Shipment Items (LIPS + VTTP)

```
lips_clean = lips.rename(columns={
    'Delivery Number': 'delivery_number',
    'Item Number': 'item_number',
    'Material Number': 'material_number',
    'Delivered Quantity': 'delivered_quantity',
    'Net Price': 'net_price',
    'Delivery Status': 'delivery_status',
    'Customer ID': 'customer_id',
    'Sales Document': 'order_id',
    'Sales Item': 'sales_item',
    'Delivery Date': 'delivery_date'
})

vttp_clean = vttp.rename(columns={
    'Shipment Number': 'shipment_number',
    'Item Number': 'item_number',
    'Material Number': 'material_number',
    'Shipped Quantity': 'shipped_quantity',
    'Item Status': 'item_status',
    'Delivery Number': 'delivery_number',
    'Customer ID': 'customer_id',
    'Sales Document': 'order_id',
    'Sales Item': 'sales_item',
    'Shipment Date': 'shipment_date'
})
```

Transform Customers (KNA1 → DW_Customers)

```python
dw_customers = kna1.copy()

dw_customers = dw_customers.rename(columns={
    'Customer ID': 'customer_id',
    'Customer Name': 'customer_name',
    'Country': 'country',
    'Region': 'region',
    'City': 'city',
    'Postal Code': 'postal_code',
    'Street Address': 'street_address',
    'Phone Number': 'phone_number',
    'Email Address': 'email',
    'Language': 'language',
    'Tax Number': 'tax_number',
    'Customer Group': 'customer_group',
    'Sales Organization': 'sales_organization',
    'Distribution Channel': 'distribution_channel',
    'Division': 'division'
})

dw_customers = dw_customers.drop_duplicates(subset=['customer_id'])
dw_customers = dw_customers.reset_index(drop=True)

dw_customers.head()
```

	customer_id	customer_name	country	region	city	postal_code	street_address	p
0	CUST001	Customer A	US	NY	New York	10001	123 Park Avenue	1
							Hauptstrasse	

Transform Customers (KNA1 → DW_Customers)

```python
dw_customers = kna1.copy()

dw_customers = dw_customers.rename(columns={
    'Customer ID': 'customer_id',
    'Customer Name': 'customer_name',
    'Country': 'country',
    'Region': 'region',
    'City': 'city',
    'Postal Code': 'postal_code',
    'Street Address': 'street_address',
    'Phone Number': 'phone_number',
    'Email Address': 'email',
    'Language': 'language',
    'Tax Number': 'tax_number',
    'Customer Group': 'customer_group',
    'Sales Organization': 'sales_organization',
    'Distribution Channel': 'distribution_channel',
    'Division': 'division'
})

dw_customers = dw_customers.drop_duplicates(subset=['customer_id'])
dw_customers = dw_customers.reset_index(drop=True)

dw_customers.head()
```

	customer_id	customer_name	country	region	city	postal_code	street_address
0	CUST001	Customer A	US	NY	New York	10001	123 Park Avenue

4. EXPLORATORY DATA ANALYSIS (EDA) AND DATA WAREHOUSE DESIGN

4.1 Exploratory Data Analysis

EDA was conducted in two layers:

4.1.1 Structural EDA

- ✓ Row count verification
- ✓ Key uniqueness checks
- ✓ Referential validation
- ✓ Null value analysis

4.1.2 Business EDA

- ✓ Orders per customer analysis
- ✓ Items per order distribution
- ✓ Shipment per order analysis
- ✓ Carrier utilization distribution
- ✓ Processing time distribution

Exploratory Data Analysis (EDA)

```python
#High-Level Summary
print("Orders:", len(dw_orders))
print("Order Items:", len(dw_order_items))
print("Shipments:", len(dw_shipments))
print("Shipment Items:", len(dw_shipment_items))
print("Customers:", len(dw_customers))
```

```
Orders: 22
Order Items: 23
Shipments: 20
Shipment Items: 21
Customers: 30
```

```python
#Unique Counts
dw_orders['order_id'].nunique()
```

```
22
```

```python
dw_customers['customer_id'].nunique()
```

```
30
```

```python
dw_shipments['shipment_number'].nunique()
```

```
20
```

```python
#Orders per Customer
orders_per_customer = dw_orders.groupby('customer_id').size()

orders_per_customer.describe()
```

```
count    22.0
mean      1.0
std       0.0
min       1.0
25%       1.0
50%       1.0
75%       1.0
max       1.0
dtype: float64
```

```python
#Items per Order
items_per_order = dw_order_items.groupby('order_id').size()

items_per_order.describe()
```

```
count    22.000000
mean      1.045455
std       0.213201
min       1.000000
```

Shipment Delay Exploration

```
order_shipment = pd.merge(
    dw_orders,
    dw_shipments,
    on='order_id',
    how='inner'
)

order_shipment['processing_time'] = (
    order_shipment['shipment_date'] - order_shipmen
).dt.days

order_shipment['processing_time'].describe()
```

```
count    20.000000
mean      7.200000
std       0.615587
min       7.000000
25%       7.000000
50%       7.000000
75%       7.000000
max       9.000000
Name: processing_time, dtype: float64
```

4.2 Staging Layer Implementation

Raw data was first loaded into staging tables (STG_* tables).

Purpose:

- Preserve original source data
- Enable auditability

- Ensure rollback capability
- Improve data reliability

LOADING PHASE:

```python
from sqlalchemy import create_engine

server = "localhost"
database = "ShopX_DW"

engine = create_engine(
    f"mssql+pyodbc://@{server}/{database}"
    "?driver=ODBC+Driver+17+for+SQL+Server"
    "&trusted_connection=yes"
)
```

```python
with engine.connect() as conn:
    print("Connected successfully ✅")
```

```
Connected successfully ✅
C:\Users\USER\AppData\Local\Temp\ipykernel_8088\3825709233.py:1: SAWarning:
  with engine.connect() as conn:
```

```python
dw_customers.to_sql(
    name='DW_Customers',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Customers Loaded ✅")
```

```
DW_Customers Loaded ✅
```

```python
dw_orders.to_sql(
    name='DW_Orders',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Orders Loaded ✅")
```

```
DW_Orders Loaded ✅
```

```python
dw_order_items.to_sql(
    name='DW_Order_Items',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Order_Items Loaded ✅")
```

```
DW_Order_Items Loaded ✅
```

```python
dw_carriers.to_sql(
    name='DW_Carriers',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Carriers Loaded ✅")
```

```
DW_Carriers Loaded ✅
```

```
DW_Carriers Loaded ✅
```

```
dw_shipments.to_sql(
    name='DW_Shipments',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Shipments Loaded ✅")
```

```
DW_Shipments Loaded ✅
```

```
dw_shipment_items.to_sql(
    name='DW_Shipment_Items',
    con=engine,
    if_exists='replace',
    index=False
)

print("DW_Shipment_Items Loaded ✅")
```

```
DW_Shipment_Items Loaded ✅
```

4.3 Data Warehouse Design – Star Schema

A star schema architecture was implemented.

Fact Tables

- ✓ DW_Delivery_Analytics
- ✓ DW_Shipment_Items

- ✓ DW_Order_Items

Dimension Tables

- ✓ DW_Customers
- ✓ DW_Carriers
- ✓ DW_Orders
- ✓ DW_Shipments

Benefits

- ✓ Improved query performance
- ✓ Simplified Power BI modeling
- ✓ Reduced complexity
- ✓ Enhanced KPI calculation efficiency

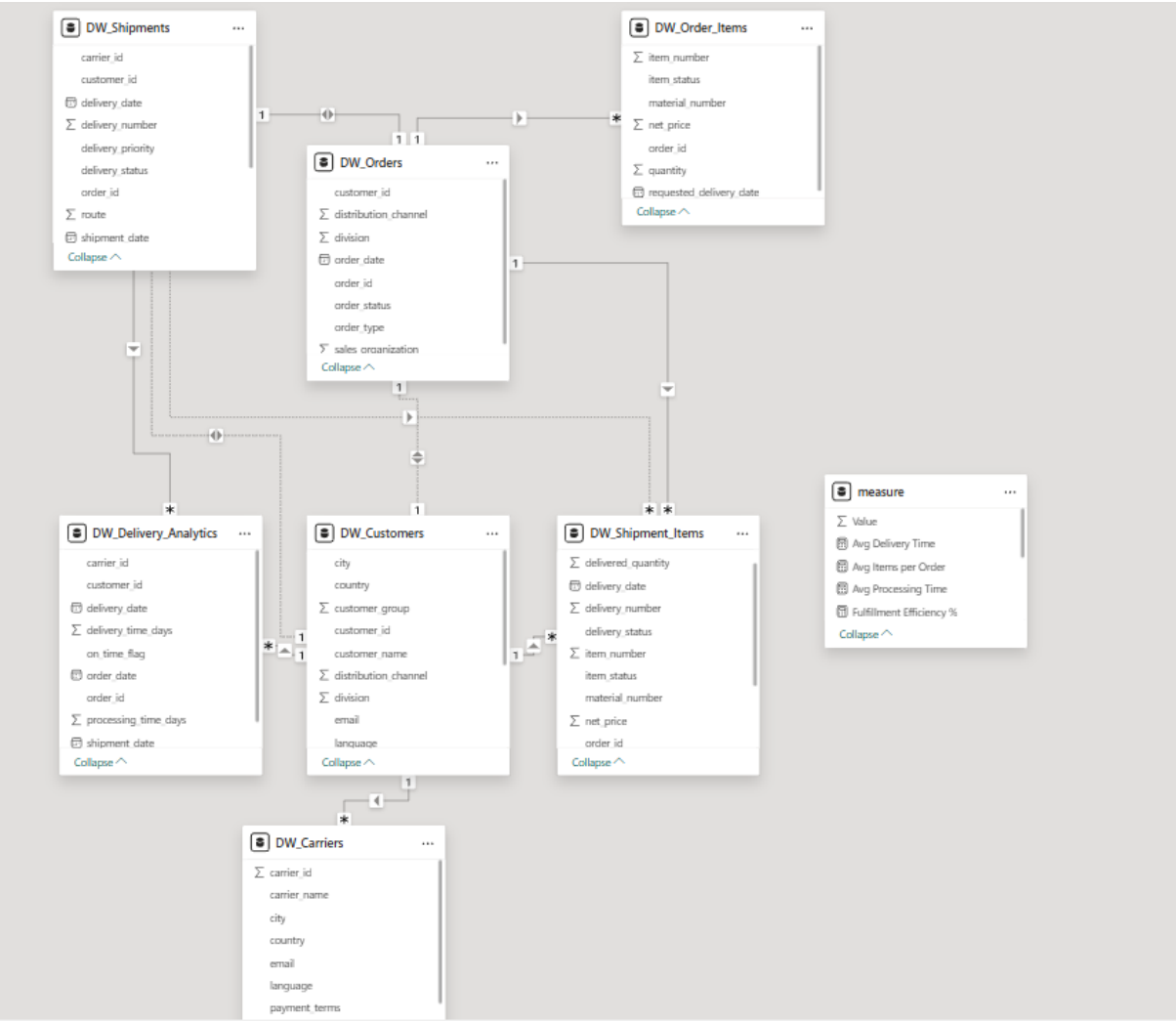
5. POWER BI IMPLEMENTATION

5.1 Data Modeling

Relationships established:

- ✓ Orders → Customers
- ✓ Orders → Shipments
- ✓ Shipments → Shipment Items
- ✓ Orders → Delivery Analytics
- ✓ Carriers → Shipments

Cardinality and filter directions were carefully configured to avoid circular dependencies.



5.2 KPI Measures (DAX)

Key DAX Measures Created:

- ✓ Total Orders
- ✓ Total Shipments
- ✓ Fulfillment Efficiency %
- ✓ Processing SLA %
- ✓ Shipment per Order Ratio
- ✓ Avg Processing Time
- ✓ Avg Items per Order
- ✓ On-Time Delivery Rate %

- ✓ Processing Delays
- ✓ Orders with Multiple Shipments

5.3 Data Cleaning in Power BI

- ✓ Corrected data types
- ✓ Standardized percentage formats
- ✓ Removed unused columns
- ✓ Verified measure logic
- ✓ Validated aggregation behavior

5.4 Dashboard Development

Page 1 – Executive Overview

High-level operational snapshot.

Page 2 – Fulfillment & Processing Analysis

Warehouse efficiency and SLA compliance.

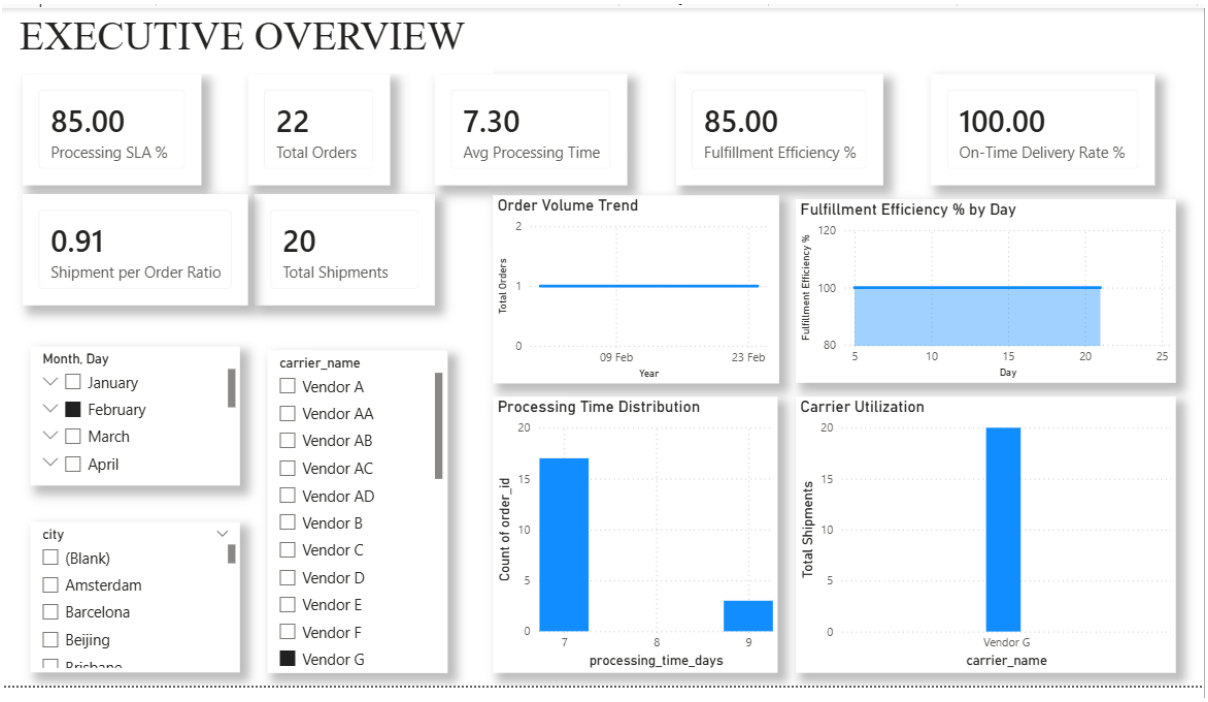
Page 3 – Order Complexity & Operational Structure

Order splitting and basket analysis.

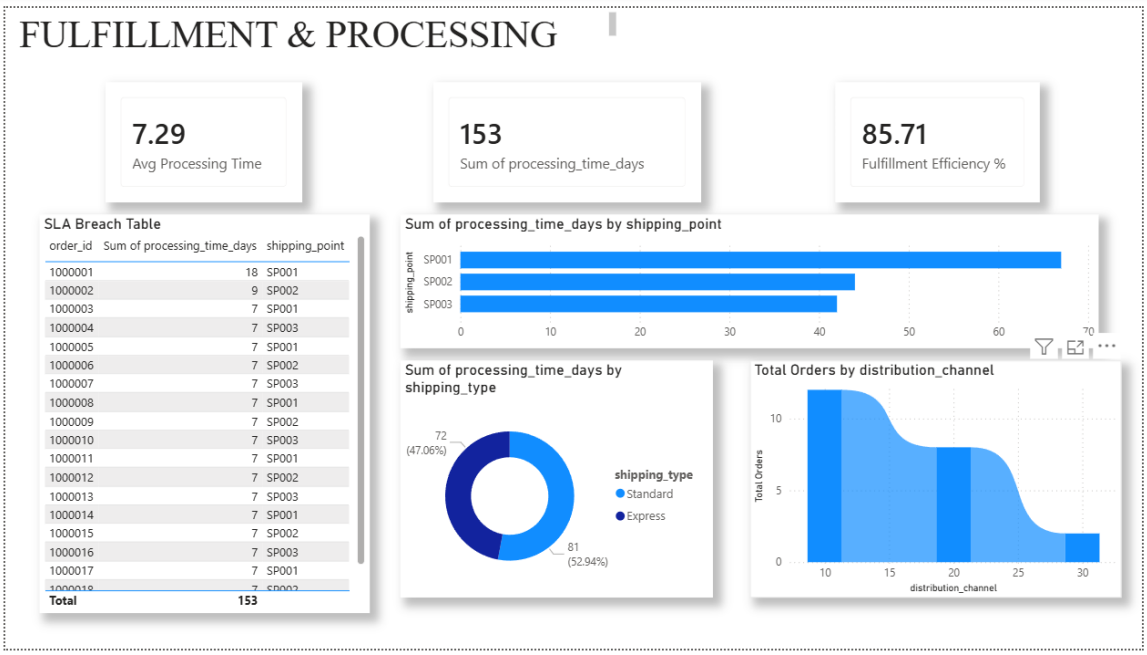
Page 4 – Customer & Sales Insights

Customer segmentation and regional performance.

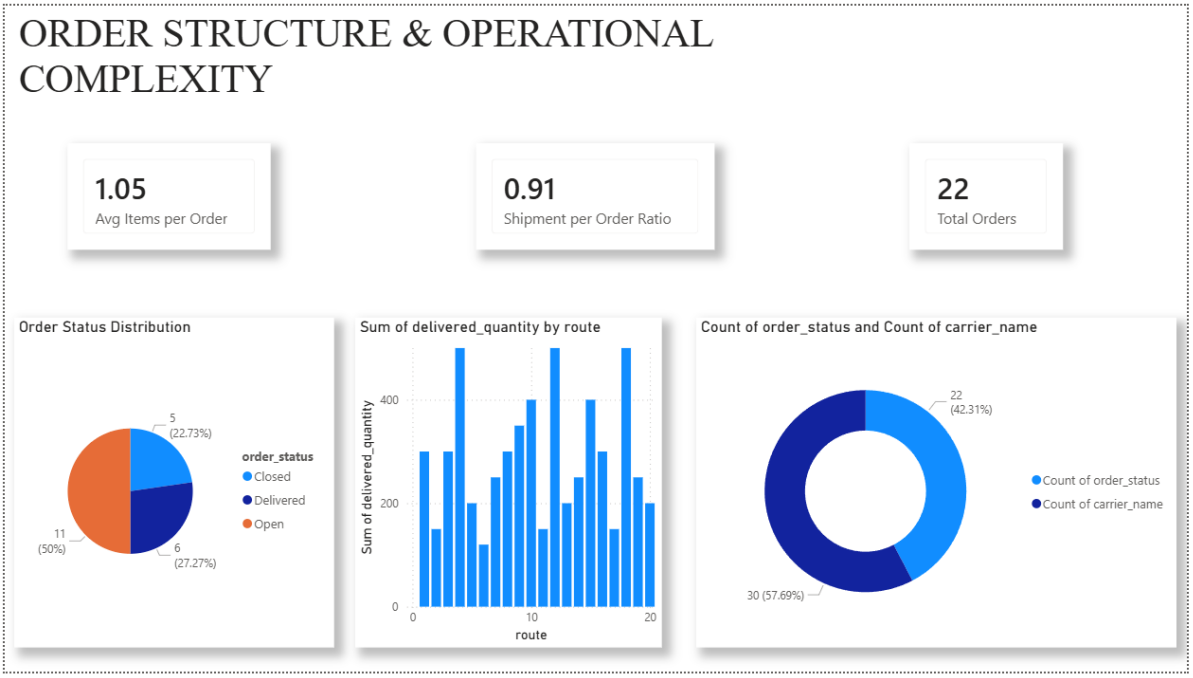
EXECUTIVE OVERVIEW

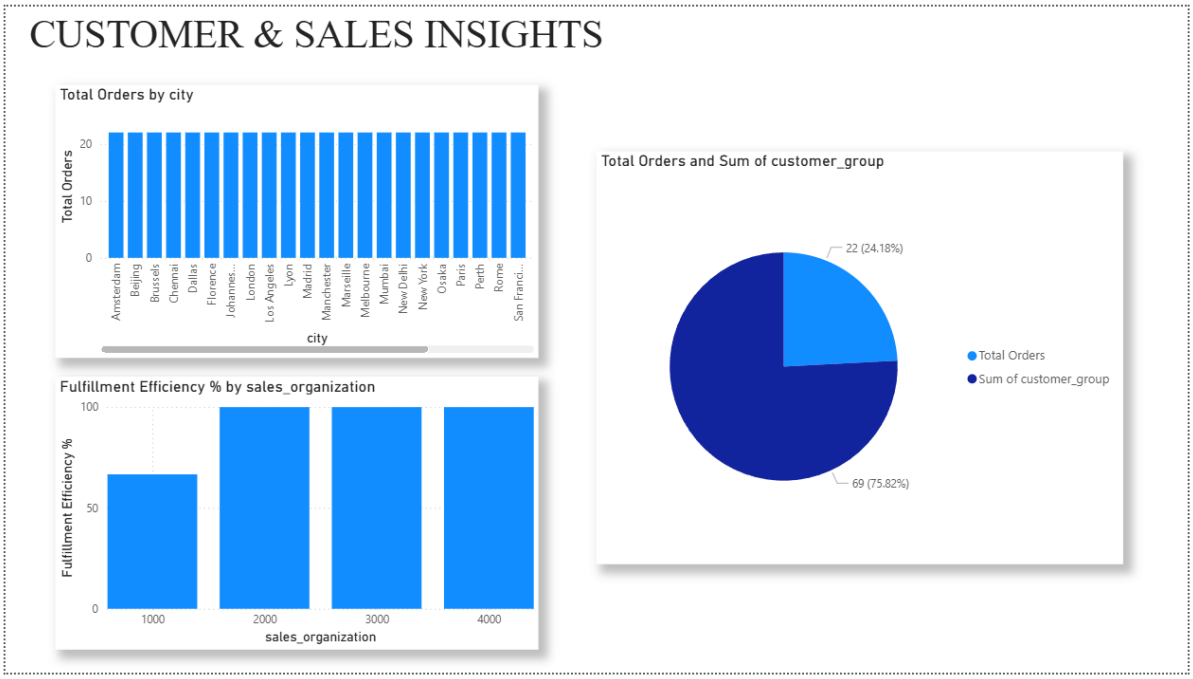


FULFILLMENT & PROCESSING



order_id	Sum of processing_time_days	shipping_point
1000001	18	SP001
1000002	9	SP002
1000003	7	SP001
1000004	7	SP003
1000005	7	SP001
1000006	7	SP002
1000007	7	SP003
1000008	7	SP001
1000009	7	SP002
1000010	7	SP003
1000011	7	SP001
1000012	7	SP002
1000013	7	SP003
1000014	7	SP001
1000015	7	SP002
1000016	7	SP003
1000017	7	SP001
Total	153	





6. DATASET CHALLENGES AND RESOLUTIONS

S.No	Challenge	Impact	Resolution
1	Delivery Date = Shipment Date	Zero delivery duration	Removed Avg Delivery Time KPI
2	Duplicate composite keys	Double counting	Applied composite validation

3	Date columns as object type	Incorrect calculations	Converted to datetime
4	Multiple shipments per order	KPI distortion	Introduced Shipment per Order Ratio
5	Missing foreign key references	Broken joins	Performed referential validation
6	Inconsistent naming	Modeling confusion	Standardized naming convention

7. PROJECT OUTCOME

The final solution delivered:

- ✓ Automated ETL pipeline using Python
- ✓ Structured SQL Server Data Warehouse
- ✓ Star schema data model
- ✓ Business-ready KPI layer
- ✓ 4-page executive Power BI dashboard
- ✓ Operational intelligence framework

The solution enables ShopX to:

- ✓ Monitor fulfillment performance
- ✓ Track SLA compliance
- ✓ Analyze operational complexity
- ✓ Optimize carrier utilization
- ✓ Improve data-driven decision making

8. CONCLUSION

The solution bridges the gap between SAP transactional data and strategic analytics, enabling improved operational transparency and performance optimization.